CSC 4100 Project 2

Cooperative Multitasking

Skills you will learn

- Creating processes
- Task switching

A new version of libos.a: libos.a (included in the above download or arm-os.tar.gz).

Cooperative Multitasking

Description

Introduction

For this assignment, you will add cooperative multitasking to your OS. In particular, you will add processes and an function for switching between processes.

The main driver

As a first step toward time slicing, you will first implement task switching via a function call. In a future assignment, you will modify your OS to use a timer to invoke the task switching function. However, for now you will create a `dispatch()` function that is callable from your process code that will switch to another process.

Below is the modified pseudocode for the `main()` from the previous assignment. Note the lines with an arrow, which are modifications to the main. The first arrow line is "`call q_init(Ready_q)`" so that your ready queue that will order the execution of the processes is initialized properly. This function, which you will write in C/C++, will set the head and tail pointers of `Ready_q` to `NULL` (or zero). The second arrow line is a call to `spawn_process(p1)`, which will set up an initial context for the process, allocate its PCB, and put a pointer to the PCB in the ready queue. The argument `p1` is a pointer to a function. This will be the code that the new process will execute.

The next arrow line after the comments is a call to `dispatch_select()`. This routine will dequeue the pointer to the process whose PCB is at the head of the `Ready_q`. You will declare a global variable called `Running` which is a pointer to a PCB. You will set it to the dequeued pointer. This Running variable will represent which process currently is running on the processor (which process is in the RUNNING state). The next line with an arrow is an inlined assembly language instruction (only use inline assembly this once; all you other assembly code should be in an assembly file). This instruction jumps to assembly code that dispatches the process indicated by the variable `Running` by restoring the process' context and jumping to its first instruction, which completes system startup. Later in this assignment description, you will be shown exactly what actual code you need to have for `spawn_process()` and `dispatch_select()` and `restore_context`.

```
int main() {

    clear the screen
    print "Running processes" on the first line of the screen

→   call q_init(Ready_q)    // initialize Ready queue data structure
                            // The prototype for q_init is: void q_init(Queue_t *)
→   set retval to the return value of spawn_process(p1)
                            // The prototype of sqawn_process is: int spawn_process(int (*pfun)())
→   check if error (retval != 0)
    // repeat the above for process p2, p3, and p4

    // Now begin running the first process ...

→   asm volatile("b restore_context");  // dispatch the current Running process
                                        //   This restores the context of the process
                                        //   at which Running points, and then jumps
                                        //   to its next instruction.
                                        // Note: retore_context is a label
                                        //       in assembler.  You must
                                        //       declare it as .global in
                                        //       the assembly file to use
                                        //       it here.
    // Note that you never return here
}
```

Figure 1: Main Driver

Ready Queue

You will have to write your own enqueue() and dequeue() functions that will be used by your OS code for the ready queue. The prototypes should look like the following:

```
void q_enqueue(PCB_Q_t *q, PCB_t *pcb);
PCB_t *q_dequeue(PCB_Q_t *q);
```

Follow the above specifications exactly, or you will not get full credit for you implementation. Likewise, your enqueue and dequeue will be implemented using a linked list structure, *not* array based. An array based version will be difficult to debug in later assignments. You should be able to determine the `PCB_t` struct and `PCB_Q_t` structs on your own (see `spawn_process()` code below). The structure must have a member named `sp` which is a `uint64_t` (`#include <stdint.h>` to get this typedef), followed by an `uint32_t` member called `pid` (short for process ID).

You can use code that you developed in your data structures course, but remember, **you cannot call `malloc()` or `new()` to create new nodes**. The professor suggest that you add a `next` pointer inside your `PCB_t` so that you do not have to wrap the pcb in a node type in order to link it into your queue. Also remember that you cannot call ANY library routines. So, if you call `cout` or `printf()`, for example, in your queue code, then you will have to remove it, because **you cannot call C/C++ library routines**.

Creating Processes

Next, you will create a function that creates processes called `spawn_process()`. This routine should initialize the process's stack, allocate a PCB, and initialize PCB with the stack top. Here is the prototype along with the pseudocode for this function:

```
int spawn_process(int (*code_address)()) {
// code_address is the pointer to the function that contains the process code


    set stackptr to return value of calling alloc_stack() to allocate a stack
          that is 1024 uint64_t in size
    check to make sure stackptr is not null
            (if it is, return -1)

    // First, set up the stack, pretend sp is a uint64_t *

    set sp to stackptr + 1024 to put sp at the
          bottom of the stack    // Note that if stackptr is a pointer to a
                                 // uint64_t, then stackptr+1024 will point
                                 // to the right place

    // create context that will set gen purpose regs to 0
    loop i from 0 to 32 (inclusive)
        decrement sp  // sp should be a pointer to a uint64_t
            set the uint64_t at which sp points to 0

            set the value at which sp+30 points to code_address // type cast code_address
                                                                // to a uint64_t so
                                                                // that the compiler does not complain)
    // Note: you will have to create the pcb struct for the following code
    set pcb to the return value of alloc_pcb() to allocate a pcb for the
            process
            check the pcb and if is null then return -1
    set the pcb's sp member to sp.
    set the pcb's pid member to next_pid  //Note that you can keep track
                                          // of the next pid using a
                                          // global variable

    enqueue the pcb onto the Ready_q  // Ready_q should be a global

    return 0 /* no errors occured */
    // Note that if alloc_stack() or alloc_pcb() returns NULL, you should return
    //  -1 instead to indicate the error
}
```

Figure 2: create_process()

So, what does `spawn_process()` do in detail? It sets up a stack for the process and enqueues it on the `Ready_q`. You will be doing simple stack based process switching, so the stack starts with initial values for the process's registers (all zeros), including the PSTATE. Also it pushes the value that control what code is executed when the process is started.

Code to start the first process running

Now that you have your processes created, you need to finish starting the system. In other words, you need to start the first process. To do so, you will write a function called `dispatch_select()` and another called `dispatch_restore`. You will call them in your `main()` (see pseudo code for `main()` above). These routines will dequeue the first process off of the ready queue and make it the current process, restore the state of the first process (which is pushed on the processes stack), and start executing at the process's first instruction. Here's the pseudocode for this function:

```
void dispatch_select() {
        set Running to the return value from dequeuing
      from Ready_q
    // Note: You should put some error code here that
    //       checked if you are dequeuing from a
    //       empty queue
}
```

Figure 4: the despatch_select() function that starts the first process.

Remember that, in the main, after calling this function, the main simply jumps (using inline assembly) to a label called `restore_context`. The `restore_context` label begins the bottom half of the `dispatch()` function, which accomplishes all that

you need in order for the first process to start. We will cover the `restore_context` label in the next section.

Code that switches between processes

The next thing that you need to do is create a function that can switch between processes. This function will save the context of the current process on the process's stack, enqueue that process, dequeue the next process and make it current, and restore that process's context. Additionally, you should have a global variable called "Running" that is a pointer to a `PCB_t`. This variable is what your OS will use to track which process is currently in the running state. Following is pseudocode for this function:

```
; Note: dispatch must be written in assembly.
 dispatch:
 1)  subtract 16*16 from the stack
 2)  copy all registers to the stack the stack
                    (in the order x0,x1,...,x30)
 3)  put the PSTATE into the stack with the
                    following code:
                       mrs     x0, SPSR_EL1      // Save PSTATE
                       str     x0, [sp, 16*15+8]
     4)      store the new sp value into the pcb where Running points
                    (note that you cannot move sp into memory directly,
          you have to do something like:
          adr     x1, Running
          ldr     x1, [x1]  // put the first value in the PCB (sp) into x1
              mov     x4, sp    // We can't str sp directly, so put sp into x4...
              str     x4, [x1]  //    Then store the new sp in x4 to the PCB
     5)      call enqueue passing it address of ReadyQ and the value of
                    Running
 restore_context:   // Note that this is where the inline assembly in main jumps
 6)  call dispatch_select()   // The function written in C/C++
 7)  read the stack pointer value out of the pcb at which Running points and put
             it into the sp register. The code should look
         like the following:
             adr             x1, Running
             str             x0, [x1]   // store the pointer to the PCB into the Running variable
                     ldr             x2, [x0]   // x2 now has the value of the stack ptr (we cant ldr sp directly)
                     mov             sp, x2     // sp now has the top of stack for the chosen process
 8)  restore the PSTATE stored in the stack
         ldr     x0, [sp, 16*15+8]   // Restore PSTATE
             msr     SPSR_EL1, x0
 9) copy all register values for x0-x30 from the stack at
         which sp points into those registers.
 10) add 16*16 to sp
 11) return
```

Figure 5: dispatch()

Note that the above code defines the label restore_context that the assembler instruction in the main() will need. The code after restore_context dequeues the first process off of the ReadyQ, restores its context, and jumps to its code, which is exactly what main() needs to do!

Code for the processes

Now you are almost ready to test. As shown in the pseudocode for main() above, your main() will create 2 processes by calling create_process two times, passing each call the address of a function to execute. So, for each process, write a function. Here's some example pseudocode for your functions:

```
// Note: implement a brute force prime checker called is_prime that takes a unsigned long long
//       that checks if the number is prime. Use division up to half
//       the number to check for primeness (you could do square root if you use
//       the math library, but then you have to link against it (-lm).
//       IN the prime checker loop, call dispatch() to make sure your processes switch.
int p1()
begin
  print a box from row 9, column 23 to row 11, column 39
  call print_to on message to print at row 10 column 25
            // Note define message as: char message[] = "Process 1: 0";
  set num to 1
  set count to 0
  loop forever
    if num is prime then
                     add one to count
          set countc to count + '0' // convert is to a character
          set message[11] to the countc
             print the message at row 10 column 25
    add one to num
    if count is greater than 9, set count to zero
    if num is greater than 4 billion
                    set num to 1
  end loop
```

```
end


int p2()
begin
     // The same for p1, but print "Process 2: 0"
     // and the box should be from row 13, column 23
     //    to row 15, column 39
     // Print the values inside that box.
end

    // Now create two more functions for p3 and p4, but begin
    // printing at column 49
    // See the video for how the running OS should look.
```

Figure 6: Processes 1 and 2.

As a final note, put your `go()` and `dispatch()` functions in a file called **process_asm.S**, put your `create_process()` function in a file called **process.c**, and your queueing functions in the file **queue.c**. Also, make the needed changes to your **Makefile** to compile and link these files into **kernel.elf**

Below is a video of the professor's running OS:

Submission

To submit, run "make clean", then create your tarball with the following command:

```
tar -czf your_tech_username_p3.tar.gz your_tech_username
```

Where **`your_tech_username`** is your TnTech email without the "`@tntech.edu`". Your project directory that contains all of your code and the **Makefile** should be in a directory whose name is the same as your TnTech user name. Remember, the professor will not grade your assignment without a properly working Makefile. Submit your assignment via iLearn.